Metody programowania równoległego

Sortowanie kubełkowe przy użyciu OpenMP - sprawozdanie

Ernest Szlamczyk, Jakub Rękas

8 Maj 2026

Spis treści

1 Wstęp **3**
1.1 Sortowanie kubełkowe 3

2 Tworzenie losowych liczb **4**
2.1 Implementacja 4
2.2 Wyniki 5
2.3 Wybór harmonogramowania 7
2.4 Prawo Amdahla 8

3 Poszczególne warianty implementacji algorytmu **8**
3.1 Struktura `Bucket` i moduł `bucket.c` 8
3.2 `main.c` - punkt wejścia i parametryzacja 9
3.3 Wariant 2 10
3.3.1 Opis algorytmu 10
3.3.2 Implementacja 10
3.4 Wariant 3 11
3.4.1 Opis algorytmu 11
3.4.2 Implementacja 12

4 Uruchamianie na Aresie **13**
4.1 Skrypty benchmarkowe 13
4.1.1 `scripts/run_bs2.sh` i `scripts/run_bs3.sh` 13
4.1.2 `scripts/run_bs2_short.sh` i `scripts/run_bs3_short.sh` 13
4.1.3 `scripts/random_tests.sh` 14

5 Wyniki **14**
5.1 Skrypty wynikowe 15
5.1.1 Format CSV 15
5.1.2 Narzędzia pomocnicze 15
5.2 Prezentacja wyników poszczególnych wariantów 15
5.2.1 Wariant 2 16
5.2.2 Wariant 3 18
5.3 Porównanie wariantów i wnioski końcowe 19

1 Wstęp

Celem tego sprawozdania jest przedstawienie wykorzystania `OpenMP` - bazowanego na dyrektywach API stworzonego do tworzenia równoległych programów na współdzielonych architekturach pamięciowych. W tym sprawozdaniu będzie badany wpływ różnych funkcji `OpenMP` na performance poszczególnych części algorytmu **sortowania kubełkowego** (*ang. bucket sort*).

W całym sprawozdaniu używamy następujących oznaczeń:

- n - liczba elementów tablicy wejściowej,
- k - liczba kubełków (w implementacji `n_buckets`; przy domyślnych parametrach $k = n$),
- p - liczba wątków OpenMP (w implementacji `n_threads = omp_get_max_threads()`),
- c - pojemność startowa pojedynczego prywatnego kubełka (w implementacji $c = 10$).

1.1 Sortowanie kubełkowe

Sortowanie kubełkowe (*ang. bucket sort*) jest algorytmem sortowania działającym w czasie liniowym dla danych o jednostajnym rozkładzie. Dane wejściowe dzielone są na k kubełków, z których każdy odpowiada pewnemu przedziałowi wartości. Algorytm składa się z czterech faz:

1. **(b) Rozdział** - każdy element tablicy trafia do kubełka odpowiadającego jego wartości.
2. **(c) Sortowanie kubełków** - każdy kubełek sortowany jest niezależnie dowolnym algorytmem porównawczym (np. quicksort).
3. **(d) Wyznaczenie pozycji zapisu** - dla każdego kubełka obliczana jest pozycja startowa w tablicy wynikowej (prefix sum rozmiarów kubełków).
4. **(d) Zapis** - posortowane kubełki przepisywane są kolejno do tablicy wyjściowej.

Złożoność przy k kubełkach i n elementach opisywana jest przez Tabele 1, natomiast Tabela 2 opisuje możliwość zrównoleglenia każdej z faz algorytmu.

Przypadek	**Złożoność**
Oczekiwana (rozkład jednostajny, $k = n$)	$O(n)$
Pesymistyczna (wszystkie elementy w jednym kubełku)	$O(n \log n)$

Tabela 1: Złożoność algorytmu bucket sort

Faza	**Niezależność iteracji**	**Możliwość zrównoleglenia**	**Uwaga**
(b) Rozdział	Nie, ponieważ istnieje sytuacja gdzie wiele wątków może pisać do tego samego kubełka	Tak, ale wymaga synchronizacji	Konieczne blokady lub eliminacja współdzielenia
(c) Sortowanie kubełków	Tak - kubełki są od siebie niezależne	Tak	Przy $k = n$ kubełki są niemal równe, więc `schedule(static)` wystarczy; dynamiczne byłoby korzystne przy mocno nierównych rozmiarach
(d) Prefix sum	Nie - każdy termin zależy od poprzedniego	Zasadniczo sekwencyjne	$O(k)$ - koszt pomijalny wobec pozostałych faz
(d) Zapis	Tak - każdy kubełek trafia do rozłącznego fragmentu tablicy	Tak	Brak wyścigu przy zapisie

Tabela 2: Analiza zrównoleglenia faz algorytmu

Wąskim gardłem jest **faza rozdziału** - jedyna wymagająca synchronizacji przy równoległym wykonaniu. Dalsze sekcje opisują dwie strategie jej zrównoleglenia: ochronę kubełków blokadami (wariant 2 3.3) oraz eliminację współdzielenia przez prywatne kubełki per wątek (wariant 3 3.4).

2 Tworzenie losowych liczb

Rozdział ten opisuje proces, oraz wyniki generowania liczb losowych w języku C przy użyciu równoległości dostarczanej przez *OpenMP*

2.1 Implementacja

Tablica wejściowa wypełniana jest liczbami pseudolosowymi z przedziału $[0, n)$ przy użyciu funkcji `erand48`, która umożliwia generowanie liczb pseudolosowych z rozkładu jednostajnego [[1]]. Każdy wątek inicjalizuje własny, niezależny stan generatora (`xsubi[3]`) entropią systemową pobraną przez `getentropy` - dzięki temu wątki nie współdzielą żadnego stanu i nie wymagają synchronizacji.

```
void random_array(double* arr, size_t n_elems, double max) {
    #pragma omp parallel
    {
```

```
        unsigned short xsubi[3];
        getentropy(xsubi, sizeof(xsubi));

        #pragma omp for schedule(static)
        for (size_t i = 0; i < n_elems; i++) {
            arr[i] = erand48(xsubi) * max;
        }
    }
}
```

Wybór harmonogramowania (`schedule`) ma tu istotny wpływ na wydajność, ponieważ każda iteracja ma identyczny koszt - losowanie jednej liczby przez `erand48`. Przebadano pięć wariantów, które zostały opisane w Tabeli 3

Wariant	**Opis**
`static(N/T)`	Statyczne, chunk $= n/p$ (jawne, obliczone poza regionem)
`static(default)`	Statyczne, chunk domyślny (kompilator/runtime dzieli równomiernie)
`static(1)`	Statyczne, chunk $= 1$ (round-robin po iteracjach)
`dynamic(N/T)`	Dynamiczne, chunk $= n/p$
`dynamic(default)`	Dynamiczne, chunk $= 1$ (domyślny dla `dynamic`)

Tabela 3: Badane warianty harmonogramowania

2.2 Wyniki

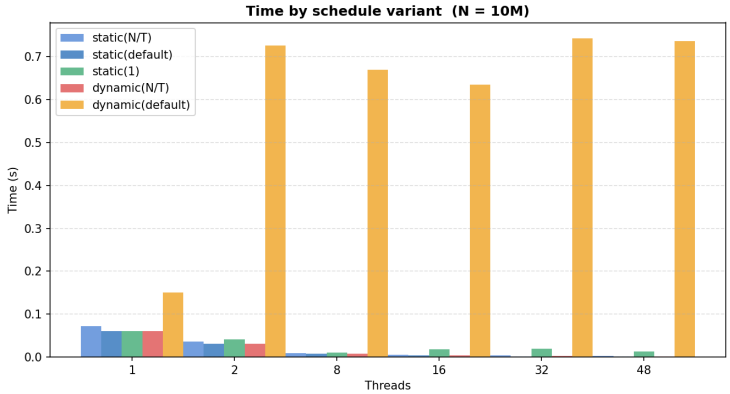
Wykres słupkowy 1 pokazuje czas wykonania wszystkich wariantów dla $n = 10\,000\,000$

`dynamic(default)` jest znacząco gorszy od pozostałych we wszystkich konfiguracjach wątków - przy chunk $= 1$ każda z 10^7 iteracji wymaga oddzielnego pobrania pracy od schedulera, co generuje ogromny narzut synchronizacji. Pozostałe cztery warianty są do siebie zbliżone.

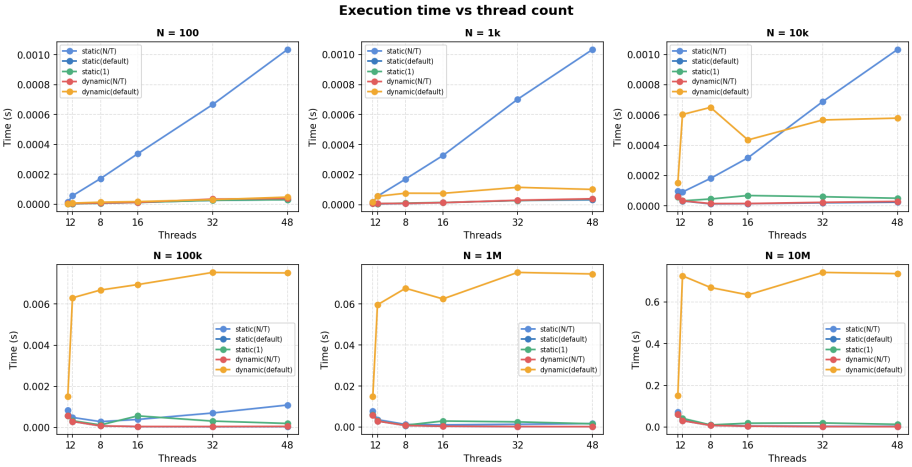
Zależność czasu od liczby wątków dla różnych rozmiarów problemu została opisana w 2

Dla małych n (100–10k) wariant `static(N/T)` zachowuje się gorzej niż pozostałe statyczne - narzut związany z wywołaniem `omp_get_max_threads()` i obliczeniem chunk poza regionem równoległym przewyższa zysk z podziału tak małej pracy. Dla dużych n (100k–10M) wszystkie warianty poza `dynamic(default)` skalują się poprawnie.

Przyśpieszenie w funkcji liczby wątków natomiast opisuje rysunek 3. Dla $n = 10\,000\,000$ warianty `static(default)`, `static(N/T)` i `dynamic(N/T)` osiągają przyspieszenie bliskie idealnemu. Wyniki dla małych n są poniżej $1\times$ - koszt



Rysunek 1: Zestawienie czasów wykonania dla różnych schedulerów

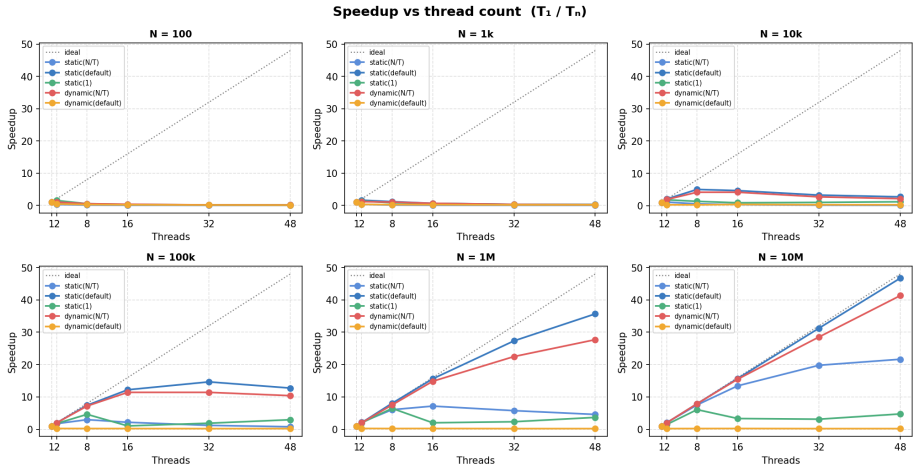


Rysunek 2: Czas wykonania w zależności od N

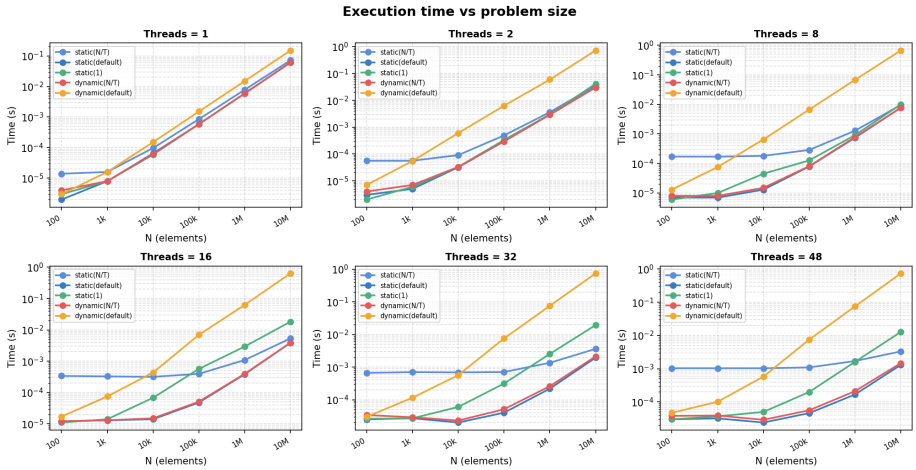
uruchomienia regionu równoległego przewyższa zysk z zrównoleglenia tak małej pracy.

Zależność czasu od rozmiaru problemu na skali logarytmicznej, dla każdej liczby wątków jest opisana na rysunku 4. We wszystkich konfiguracjach wątków czas rośnie liniowo z n (prosta linia na skali log-log), co potwierdza oczekiwaną złożoność $O(n/p)$ na wątek. `dynamic(default)` leży konsekwentnie powyżej pozostałych o stały czynnik narzutu schedulera. Warto zauważyć interesujący `static(N/T)`. Jego duży narzut w przypadku threads 48 nie osiąga momentu, w którym główną składową czasu jest ilość elementów.

Wykresy te dodatkowo bardzo dobrze pokazują wpływ narzutu na całkowity



Rysunek 3: Przyspieszenie w funkcji liczby wątków



Rysunek 4: Czas wykonania w skali log-log

czas. Jak zostało zauważone wcześniej w momencie kiedy główną składową jest narzut wykres jest bliski funkcji $f(x) = N_z$ (N_z w tym przypadku to wartość narzutu). Dopiero dla większych N osiąga on kształt podobny do funkcji $f(x) = x$.

2.3 Wybór harmonogramowania

Na podstawie wyników w finalnej implementacji (`random_array` w `random.c`) zastosowano `schedule(static)` bez jawnego chunk - OpenMP dzieli iteracje

równomiernie między wątki, co dla identycznego kosztu każdej iteracji jest optymalnym wyborem: brak narzutu dynamicznego schedulera i brak ryzyka false sharing przy małych chunkach.

2.4 Prawo Amdahla

Prawo Amdahla mówi, że dla programu z sekwencyjną frakcją f_s maksymalne przyspieszenie wynosi:

$$S(p) = \frac{1}{f_s + \frac{1-f_s}{p}} \xrightarrow{p\to\infty} \frac{1}{f_s} \tag{1}$$

W `random_array` jedyną sekwencyjną częścią jest inicjalizacja generatora przez `getentropy` (stały koszt, niezależny od n). Dla dużego n frakcja ta zanika i $f_s \approx 0$ - przyspieszenie zbliża się do idealnego. Potwierdzają to wartości dla wariantu `static(default)` dla $n = 10\,000\,000$ wyliczone w tabeli 4

p	**przyspieszenie**	**idealne**
2	1.99×	2.00×
8	7.89×	8.00×
16	15.70×	16.00×
32	31.25×	32.00×
48	46.71×	48.00×

Tabela 4: Porównanie przyspieszenia z prawem Amdahla

Prawo Amdahla jest tu dobrze zachowane. Wyniki poniżej 1× dla małych n nie są naruszeniem prawa - dla bardzo małej pracy narzut uruchomienia regionu równoległego (tworzenie wątków, synchronizacja) staje się dominujący. Amdahl modeluje przyspieszenie dla ustalonego n; dla $n \to 0$ przyspieszenie zawsze spada poniżej 1.

3 Poszczególne warianty implementacji algorytmu

Oba warianty dzielą wspólną infrastrukturę: strukturę danych kubełka, punkt wejścia programu oraz interfejs funkcji sortującej.

3.1 Struktura `Bucket` i moduł `bucket.c`

Kubełek reprezentowany jest przez prostą strukturę:

```
typedef struct {
```

```
    double* elems;    // wskaźnik na tablicę elementów
    size_t  n_elems;  // aktualna liczba elementów
    size_t  capacity; // maksymalna pojemność
} Bucket;
```

Moduł `bucket.c` udostępnia cztery operacje, które zostały wymienione w tabeli 5. Warto zauważyć, że kubełki **nie są dynamicznie rozszerzane** - `bucket_push` po prostu zwraca błąd przy przepełnieniu. Dobór pojemności jest obowiązkiem wywołującego; pojemność startowa musi uwzględniać pesymistyczny rozkład danych.

Funkcja	**Opis**
`bucket_init(capacity)`	Alokuje kubełek o zadanej pojemności
`bucket_push(bucket, elem)`	Wstawia element; zwraca `PUSH_FAILED` gdy kubełek pełny
`create_buckets(n, capacity)`	Alokuje tablicę `n` kubełków **jednym** `malloc`: ciągły blok `[Bucket[0]...Bucket[n-1] | elems[0]...elems[n-1]]`; każde `Bucket[i].elems` wskazuje odpowiedni fragment slaba
`destroy_buckets(buckets)`	Zwalnia cały blok jednym `free` (metadane i slab `elems` to jedna alokacja)

Tabela 5: Operacje na strukturze Bucket

3.2 `main.c` - punkt wejścia i parametryzacja

Program przyjmuje dwa opcjonalne argumenty: `n_elems` (domyślnie 10^6) i ścieżkę do pliku CSV z wynikami. Zakres wartości jest ustalony jako $[0, n)$ - każda liczba losowana jest przez `erand48(...) * n`, co przy $k = n$ kubełkach daje jednostajny rozkład z oczekiwaną jedną liczbą na kubełek.

```
static size_t bucket_idx(double elem) {
    return (size_t)elem;  // obcięcie do części całkowitej
}
```

Globalne `buckets` (wspólne dla obu wariantów) alokowane są z pojemnością 10 - wystarczającą przy jednostajnym rozkładzie, gdzie oczekiwana liczba elementów na kubełek wynosi 1. Pomiar `t_total` obejmuje całe wywołanie `bucket_sort`, a poprawność wyniku weryfikowana jest sekwencyjną pętlą `is_sorted` poza pomiarem czasu. Wyniki (czasy faz, flaga poprawności) dopisywane są do CSV - program zawsze dopisuje wyłącznie wiersz danych, bez nagłówka.

3.3 Wariant 2

3.3.1 Opis algorytmu

Wariant 2 zrównolegla fazę rozdziału poprzez ochronę każdego kubełka globalnego **oddzielnym lockiem**. Wątki wstawiają elementy bezpośrednio do wspólnych `buckets[]`, zabezpieczając każde wstawienie parą `omp_set_lock` / `omp_unset_lock`. Nie istnieje faza scalania ani prywatne kopie kubełków - narzut pamięciowy wynosi $O(k)$ locków, niezależnie od p.

Algorytm przebiega w czterech etapach:

1. **(a) Inicjalizacja locków** - k locków alokowanych i inicjalizowanych równolegle w pętli `parallel for` poza mierzonym czasem.
2. **(b) Rozdział** - pętla `parallel for schedule(static)` po wszystkich n elementach. Każdy element trafia do kubełka globalnego pod ochroną jego locka.
3. **(c) Sortowanie kubełków** - niezależne sortowanie każdego kubełka przez `qsort`. Brak kolizji.
4. **(d) Zapis** - wyznaczenie pozycji zapisu (prefix sum, sekwencyjnie), a następnie równoległe przepisanie posortowanych kubełków do tablicy wyjściowej blokami `memcpy`.

3.3.2 Implementacja

Synchronizacja opiera się na tablicy `omp_lock_t locks[k]`, gdzie `locks[i]` chroni wyłącznie `buckets[i]`. Faza rozdziału używa dyrektywy `#pragma omp parallel for schedule(static)`:

```
omp_set_lock(&locks[idx]);
bucket_push(&buckets[idx], array[i]);
omp_unset_lock(&locks[idx]);
```

Granularność blokady jest **per kubełek** - wątki piszące do różnych kubełków nigdy się nie blokują wzajemnie. Przy $k = n$ i rozkładzie jednostajnym kolizje na tym samym locku są rzadkie.

Poprawność Algorytm został przetestowany i działa poprawnie. Poglądowe wyniki testów znajdują się w tabeli 6. Test przeprowadzono dla $n = 100\,000$ elementów oraz $p \in \{1, 2, 8, 16, 48\}$ wątków - we wszystkich przypadkach tablica wyjściowa była poprawnie posortowana (`Sorted: yes`).

a) Ochrona danych współdzielonych Wariant 2 chroni dane przez **blokowanie per kubełek** zamiast eliminacji współdzielenia. Tablica wejściowa `array[]` jest czytana w trybie *read-only*. Każdy kubełek globalny chroniony jest przez swój lock, co wyklucza równoczesny zapis. Fazy sortowania i zapisu nie wymagają synchronizacji dzięki rozłącznemu przydzielaniu kubełków wątkom.

Tabela 6: Wyniki testów poglądowych wydajnościowych dla wariantu 2 ($n = 100\,000$).

p	t_{random} [s]	$t_{distribute}$ [s]	t_{merge} [s]	t_{sort} [s]	$t_{writeback}$ [s]	t_{total} [s]
1	0.000884	0.004624	0.000000	0.001802	0.001137	0.008142
2	0.000484	0.002775	0.000000	0.000955	0.000591	0.004610
8	0.000306	0.001439	0.000000	0.000279	0.000315	0.002128
16	0.000423	0.001117	0.000000	0.000152	0.000297	0.001648
48	0.001184	0.001231	0.000000	0.000121	0.000485	0.001962

b) Złożoność obliczeniowa i pamięciowa Złożonośc obliczeniowa i pamięciowa dla poszczególnych etapów funkcji zostały przedstawione w tabeli 7

Etap	**Span (ścieżka krytyczna)**	**Praca łączna**	**Pamięć dodatkowa**
(a) Init locków	$O(k/p)$	$O(k)$	$O(k)$ - tablica `locks`
(b) Rozdział	$O(n/p) + \text{contention}$	$O(n)$	$O(1)$
(c) Sortowanie	$O(\frac{n}{k} \log \frac{n}{k})$	$O(n \log \frac{n}{k})$	$O(1)$ na kubełek
(d) Prefix sum	$O(k)$ sekwencyjnie	$O(k)$	$O(k)$ - tablica `write_at`
(d) Zapis	$O(n/p)$	$O(n)$	$O(1)$
Całość	$O(\frac{k}{p} + \frac{n}{p} + \frac{n}{k} \log \frac{n}{k})$	$O(n \log \frac{n}{k})$	$O(k)$

Tabela 7: Analiza złożoności Wariantu 2

3.4 Wariant 3

3.4.1 Opis algorytmu

Wariant 3 eliminuje konflikty poprzez przydzielenie każdemu wątkowi **prywatnej kopii wszystkich kubełków**.

Algorytm przebiega w etapach:

1. **(b) Rozdział** - każdy wątek zapisuje do własnego wiersza kubełków (`thread_buckets`).
2. **(b') Scalanie** - podzbiór kubełków globalnych konkatenuje odpowiednie wiersze prywatne od wszystkich wątków.
3. **(c) Sortowanie kubełków** - niezależne sortowanie każdego kubełka globalnego.
4. **(d) Zapis** - równoległe przepisanie do tablicy wyjściowej.

3.4.2 Implementacja

Kluczową strukturą jest `thread_buckets` - tablica $p \cdot k$ obiektów `Bucket` w układzie *row-major*:

$$\underbrace{[b_0 \mid b_1 \mid \cdots \mid b_{k-1}]}_{\text{wątek } 0} \underbrace{[b_0 \mid b_1 \mid \cdots \mid b_{k-1}]}_{\text{wątek } 1} \cdots \underbrace{[b_0 \mid b_1 \mid \cdots \mid b_{k-1}]}_{\text{wątek } p-1} \tag{2}$$

Pojemność startowa $c = 10$ jest wystarczająca przy rozkładzie jednostajnym, gdyż oczekiwana liczba elementów w prywatnym kubełku wynosi $\frac{n}{p \cdot k}$, co dla $k = n$ daje wartość ≤ 1.

Poprawność Algorytm został przetestowany i działa poprawnie. Wyniki testów znajdują się w tabeli 8. Test przeprowadzono dla $n = 100\,000$ elementów oraz $p \in \{1, 2, 8, 16, 48\}$ wątków - we wszystkich przypadkach tablica wyjściowa była poprawnie posortowana (`Sorted: yes`).

Tabela 8: Wyniki poglądowe testów wydajnościowych dla wariantu 3 ($n = 100\,000$).

p	t_{random} [s]	$t_{distribute}$ [s]	t_{merge} [s]	t_{sort} [s]	$t_{writeback}$ [s]	t_{total} [s]
1	0.000964	0.002744	0.004035	0.001931	0.001124	0.010598
2	0.000493	0.002471	0.007361	0.001087	0.000792	0.012517
8	0.000324	0.002277	0.008166	0.000338	0.000534	0.015255
16	0.000438	0.002556	0.008314	0.000177	0.000476	0.020361
48	0.001433	0.003129	0.025899	0.000123	0.000620	0.055888

a) Ochrona danych współdzielonych Wariant 3 eliminuje współdzielenie poprzez podział przestrzeni danych. Każdy wątek operuje na rozłącznych segmentach pamięci. Jedyną zmienną wymagającą synchronizacji jest flaga `sort_failed`, zabezpieczona przez `#pragma omp atomic write`.

b) Złożoność obliczeniowa i pamięciowa Złożoność obliczeniowa i pamięciowa dla wariantu 3 została opisana w tabeli 9

> **Uwaga dotycząca złożoności pamięciowej.** Przy parametrach $n = k = 10^6$ i $p = 64$ zaalokowana pamięć wynosi:
>
> $$p \cdot k \cdot \text{sizeof(Bucket)} + p \cdot k \cdot c \cdot \text{sizeof(double)} \approx 6{,}6\,\text{GB} \tag{3}$$
>
> Dzięki pojedynczej alokacji narzut `malloc` jest pomijalny, jednak tak duże zapotrzebowanie może stać się wąskim gardłem dla systemów z małą ilością pamięci RAM.

Etap	Span (ścieżka krytyczna)	Praca łączna	Pamięć dodatkowa
(b) Rozdział	$O(n/p)$	$O(n)$	$O(p \cdot k \cdot c)$
(b') Scalanie	$O(k)$	$O(p \cdot k)$	$O(n)$ - kubełki globalne
(c) Sortowanie	$O(\frac{n}{k} \log \frac{n}{k})$	$O(n \log \frac{n}{k})$	$O(1)$
(d) Prefix sum	$O(k)$ sekwencyjnie	$O(k)$	$O(k)$
(d) Zapis	$O(n/p)$	$O(n)$	$O(1)$
Całość	$O(k + \frac{n}{k} \log \frac{n}{k})$	$O(p \cdot k + n \log \frac{n}{k})$	$O(n + p \cdot k \cdot c)$

Tabela 9: Analiza złożoności Wariantu 3

4 Uruchamianie na Aresie

Pomiary wykonano na klastrze **Ares** przy użyciu skryptów SLURM znajdujących się w katalogu `scripts/`. Każdy skrypt jest jednocześnie skryptem Bash i plikiem zadania SLURM - można go przesłać poleceniem:

```
sbatch scripts/run_bs2.sh
```

Wszystkie skrypty rezerwują **1 węzeł wyłącznie** (`-exclusive`), 1 zadanie z 48 rdzeniami (`-cpus-per-task=48`). Przed uruchomieniem pomiarów skrypt ładuje moduł `gcc` (jeśli dostępne jest polecenie `module`) i buduje pliki wykonywalne z pomocą dostarczonego `Makefile`. Wiązanie wątków do rdzeni ustawione jest przez zmienne środowiskowe `OMP_PLACES=cores` i `OMP_PROC_BIND=close`, co minimalizuje koszt migracji wątków między rdzeniami.

Wyniki dopisywane są do pliku CSV w katalogu `results/`. Nazwa pliku jest parametryzowana: gdy skrypt uruchamiany jest przez SLURM, do nazwy dodawany jest identyfikator zadania (np. `bs2_123456.csv`); lokalnie używana jest nazwa domyślna (np. `bs2.csv`).

4.1 Skrypty benchmarkowe

4.1.1 `scripts/run_bs2.sh` i `scripts/run_bs3.sh`

Pełne benchmarki odpowiednio dla wariantu 2 i 3. Parametry przestrzeni pomiarowej zostały opisane w tabeli 10. Kombinacje z $p > n$ są pomijane (zbędna konfiguracja przy tak małej pracy). Każda poprawna kombinacja uruchamiana jest raz; wielokrotne uruchomienie skryptu dopisuje kolejne wiersze do tego samego CSV, co pozwala uśrednić wyniki po stronie skryptów rysujących.

4.1.2 `scripts/run_bs2_short.sh` i `scripts/run_bs3_short.sh`

Skrócone warianty do szybkiej weryfikacji poprawności i wstępnej oceny wydajności na partycji testowej

Parametr	Wartości
n	37 wartości: 100, 300, 500, 1 000, ..., 1 000 000, 1 500 000, 2 000 000, 2 500 000
p	1, 2, 3, 4, 6, 8, 10, 12, 16, 20, 24, 32, 48
Limit czasu	1 godz. 30 min (`-time=01:30:00`)
Partycja	`plgrid`

Tabela 10: Konfiguracja pełnych benchmarków

Parametr	Wartości
n	1 000, 10 000, 100 000, 1 000 000
p	1, 4, 16, 48
Limit czasu	10 min (`-time=00:10:00`)
Partycja	`plgrid-testing`

Tabela 11: Konfiguracja krótkich benchmarków

4.1.3 `scripts/random_tests.sh`

Benchmark do pomiaru wydajności generowania losowych liczb (`random_array`) dla różnych wariantów harmonogramowania

Parametr	Wartości
n	100, 1 000, 10 000, 100 000, 1 000 000, 10 000 000
p	1, 2, 8, 16, 32, 48
Limit czasu	1 godz. (`-time=01:00:00`)
Partycja	`plgrid`

Tabela 12: Konfiguracja testów generatora losowego

Skrypt buduje i uruchamia binarny plik `scheduler`, który testuje pięć wariantów harmonogramowania opisanych w sekcji 2.

5 Wyniki

Sekcja wyniki została podzielona na trzy części:

- skrypty wynikowe,
- prezentacja wyników poszczególnych wariantów,

- porównanie wariantów między sobą.

5.1 Skrypty wynikowe

5.1.1 Format CSV

Każde uruchomienie programu dopisuje jeden wiersz do pliku CSV (bez nagłówka):

Kolumna	Typ	Opis
`algorithm`	string	nazwa wariantu (`bs2` lub `bs3`)
`n_threads`	int	liczba wątków OpenMP
`n_elems`	int	rozmiar tablicy wejściowej
`t_random`	float	czas losowania tablicy [s]
`t_distribute`	float	czas rozdziału elementów do kubełków [s]
`t_merge`	float	czas scalania prywatnych kubełków [s] (0 dla bs2)
`t_sort`	float	czas sortowania kubełków [s]
`t_writeback`	float	czas prefix-sum + przepisania do tablicy [s]
`t_total`	float	całkowity czas sortowania (mierzony niezależnie) [s]
`sorted`	int	weryfikacja poprawności (`1` = OK, `0` = błąd)

Tabela 13: Format danych wyjściowych CSV

Przykładowy wiersz:

```
bs2,8,1000000,0.012481,0.043210,0.000000,0.031540,0.005320,0.080100,1
```

5.1.2 Narzędzia pomocnicze

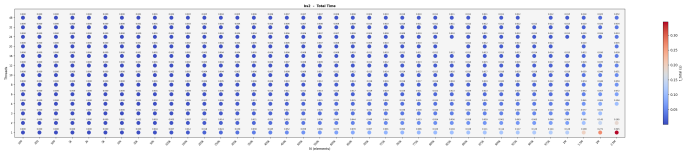
- `utils/bs_common.py`: Współdzielony moduł definiujący kolumny, fazy (`COMPONENTS`) oraz funkcje formatujące dane dla Pandas.
- `utils/plot_heatmap.py`: Generuje mapy cieplne `t_total` oraz siatkę 2×2 dla poszczególnych faz.
- `utils/plot_slice.py`: Generuje wykresy liniowe dla ustalonego N lub liczby wątków.
- `utils/plot_all_slices.sh`: Automatyzuje proces generowania wykresów dla wszystkich plików wynikowych w katalogu `results/`.

5.2 Prezentacja wyników poszczególnych wariantów

Pomiary wykonano dla $n \in [100,\ 2\,500\,000]$ (37 wartości) oraz $p \in \{1, 2, 3, 4, 6, 8, 10, 12, 16, 20, 24, 32, 48\}$.

5.2.1 Wariant 2

Wyniki pomiarów całkowitego czasu wykonania (t_{total}) dla Wariantu 2 w pełnej przestrzeni parametrów (n, p) przedstawiono w formie mapy cieplnej na rysunku 5. Analiza wizualna wskazuje na występowanie wyraźnego gradientu: czas wykonania skaluje się proporcjonalnie do rozmiaru problemu n oraz odwrotnie proporcjonalnie do liczby użytych wątków p. Ciepłe barwy, odpowiadające najdłuższym czasom obliczeń, ograniczone są do obszaru niskiej równoległości przy maksymalnych rozmiarach tablicy wejściowej. Już przy użyciu 24 wątków cała przestrzeń parametrów przyjmuje barwy chłodne (niebieskie), co świadczy o wysokiej efektywności zrównoleglenia algorytmu. (Obraz w pełnej rozdzielczości dostępny w zasobach: **plots/bs2_static/bs2_total_heatmap.png**).



Rysunek 5: Mapa cieplna całkowitego czasu wykonania (t_{total}) dla Wariantu 2.

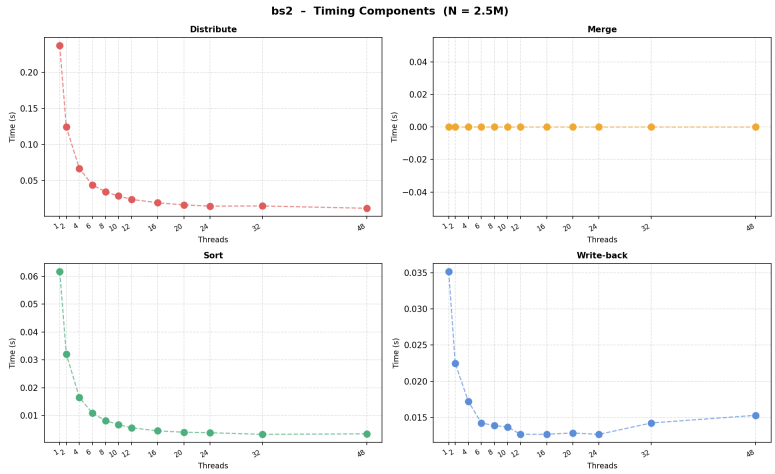
Szczegółowy rozkład czasu na poszczególne fazy algorytmu dla największego badanego problemu ($n = 2\,500\,000$) zaprezentowano na rysunku 6. Z pomiarów wynika, że dominującym komponentem jest faza **rozdziału**, co jest bezpośrednim skutkiem narzutu generowanego przez mechanizmy synchronizacji (blokady typu *lock* per kubełek). Zaobserwowano jednak korzystne skalowanie tej fazy – przy założeniu $k = n$, prawdopodobieństwo jednoczesnego dostępu wielu wątków do tego samego kubełka jest znikome, co skutecznie minimalizuje zjawisko *lock contention*. Fazy sortowania oraz zapisu wykazują skalowanie bliskie idealnemu, co wynika z pełnej niezależności operacji na rozłącznych segmentach pamięci.

Charakterystykę skalowania względem rozmiaru problemu przedstawiono na rysunku 7. Przy maksymalnej dostępnej liczbie wątków ($p = 48$), zależność czasu od n zachowuje charakter liniowy, co potwierdza utrzymanie teoretycznej złożoność $O(n)$ również w implementacji równoległej.

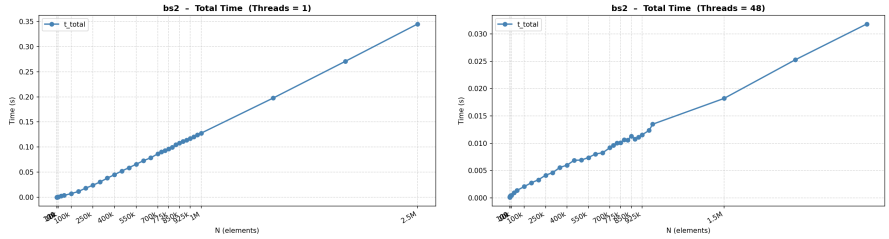
Analiza składowych czasu w całej dziedzinie, przedstawiona na rysunku 8, potwierdza, że wraz ze wzrostem liczby wątków koszt sortowania staje się marginalny, a głównym czynnikiem determinującym wydajność pozostaje faza rozdziału.

W tabeli 14 zestawiono wartości przyspieszenia dla $n = 2\,500\,000$. Odnotowano dynamiczny wzrost wydajności do poziomu $p = 24$. Powyżej tej wartości następuje nasycenie (plateau), wywołane prawdopodobnie przez narzuty komunikacyjne w architekturze NUMA klastra Ares oraz narzut związany z zarządzaniem dużą liczbą blokad.

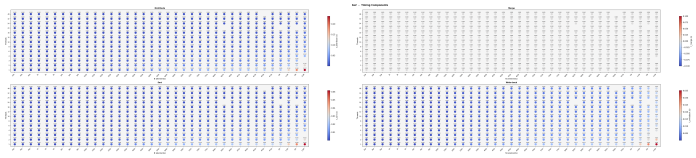
Interpretacja w oparciu o prawo Amdahla. Wyznaczona na podstawie danych eksperymentalnych frakcja sekwencyjna wynosi $f_s \approx 5{,}4\%$. Definiuje ona



Rysunek 6: Czas wykonania poszczególnych faz algorytmu dla $N = 2.5$M (Wariant 2).



Rysunek 7: Zależność t_{total} od rozmiaru N dla skrajnych konfiguracji wątkowych ($p = 1$ oraz $p = 48$).



Rysunek 8: Analiza komponentów czasu wykonania w funkcji n i p (Wariant 2).

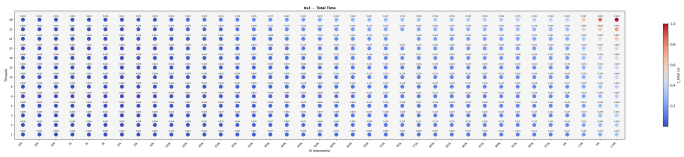
teoretyczny limit przyspieszenia na poziomie $S_{\max} \approx 18{,}6\times$. Głównym źródłem części sekwencyjnej jest operacja obliczania sum prefiksowych (prefix sum) o złożoności $O(k)$, która przy $k = 2{,}5 \cdot 10^6$ generuje mierzalny koszt, oraz stały narzut synchronizacji bariery kończącej regiony równoległe.

Tabela 14: Przyspieszenie Wariantu 2 w funkcji liczby wątków ($N = 2.5$M).

Liczba wątków (p)	**Czas** $t_{\mathbf{total}}$ **[s]**	**Przyspieszenie**
1	0.345	1.00×
4	0.104	3.32×
8	0.059	5.85×
12	0.044	7.84×
24	0.030	11.50×
32	0.034	10.15×
48	0.032	10.78×

5.2.2 Wariant 3

Analiza mapy cieplnej całkowitego czasu wykonania dla Wariantu 3, przedstawionej na rysunku 9, prowadzi do wniosków skrajnie odmiennych niż w przypadku poprzedniej implementacji. Zaobserwowano zjawisko **anty-skalowania** (ang. *anti-scaling*): czas wykonania algorytmu rośnie wraz ze zwiększaniem liczby wątków p. Gradient czasowy jest odwrócony względem oczekiwanego – najkrótsze czasy obliczeń odnotowano dla wykonania sekwencyjnego ($p = 1$).

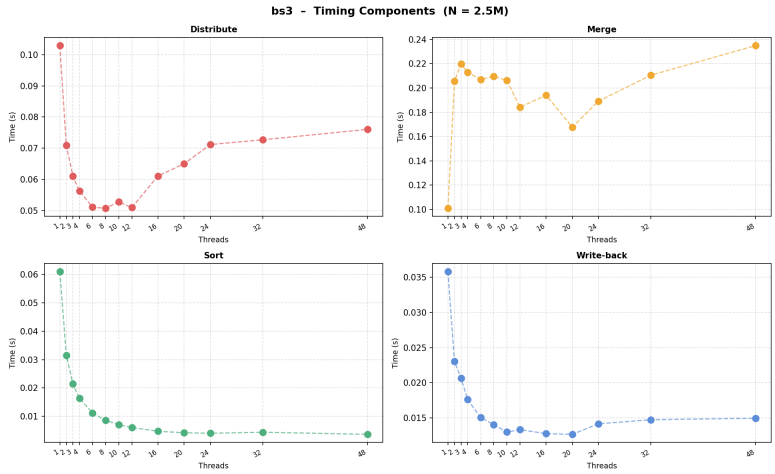


Rysunek 9: Mapa cieplna całkowitego czasu wykonania (t_{total}) dla Wariantu 3.

Identyfikację przyczyn braku efektywności umożliwia analiza komponentów czasu wykonania dla $n = 2\,500\,000$ (Rysunek 10). O ile fazy **Distribute** i **Sort** skalują się poprawnie, o tyle nowym wąskim gardłem staje się faza **Merge**. Najlepszą teorią wyjaśniającą to zjawisko jest teza, twierdząca, że przy dużej liczbie kubełków k, proces scalania prywatnych struktur wątków generuje znaczną liczbę chybień pamięci podręcznej (*cache-misses*), gdyż wątek musi odczytać dane z p rozproszonych wierszy `thread_buckets`. Dodatkowo faza **Write-back**, choć skalowalna, nie jest w stanie zniwelować narzutu generowanego przez etap scalania.

Kluczowym czynnikiem degradującym wydajność jest narzut pozamierzony, zestawiony w tabeli 15. Różnica między zmierzonym sumarycznym czasem faz a całkowitym czasem t_{total} rośnie gwałtownie wraz z p. Jest to wynik sekwencyjnej inicjalizacji struktur `thread_buckets` w funkcji `create_buckets`. Przy $p = 48$ i $k = 2{,}5$M pętla inicjalizująca musi przejść przez $120 \cdot 10^6$ obiektów `Bucket`, co generuje narzut rzędu 0,671 s, całkowicie dominując czas pracy algorytmu.

Wariant 3 fundamentalnie narusza założenia prawa Amdahla. Frakcja sekwencyjna f_s nie jest stała, lecz rośnie liniowo wraz z liczbą wątków ze względu



Rysunek 10: Rozkład czasowy faz algorytmu dla Wariantu 3 ($N = 2.5$M).

Tabela 15: Analiza narzutu sekwencyjnego i pozamierzonego w Wariancie 3 ($N = 2.5$M).

Parametr	$p = 1$	$p = 48$
Suma zmierzonych faz [s]	0.300	0.330
Całkowity czas t_{total} [s]	0.315	1.000
Niezliczony narzut (inicjalizacja) [s]	**0.014**	**0.671**

na koszt alokacji i inicjalizacji struktur prywatnych. W efekcie, zamiast dążyć do teoretycznego sufitu przyspieszenia, wydajność algorytmu drastycznie spada wraz ze wzrostem równoległości.

5.3 Porównanie wariantów i wnioski końcowe

Bezpośrednie zestawienie wydajności obu implementacji dla maksymalnego rozmiaru problemu przedstawiono w tabeli 16.

Tabela 16: Porównanie wydajności t_{total} dla $N = 2\,500\,000$.

Liczba wątków (p)	**Wariant 2 [s]**	**Wariant 3 [s]**	**Relacja (bs2 / bs3)**
1	0.345	0.315	1.10×
4	0.104	0.358	0.29×
12	0.044	0.419	0.10×
48	0.032	1.000	0.03×

Mimo że Wariant 3 wykazuje nieznaczną przewagę w teście sekwencyjnym

(dzięki całkowitemu brakowi mechanizmów blokujących), Wariant 2 okazuje się bezkonkurencyjny w środowisku wielowątkowym. Przy 48 wątkach implementacja oparta na blokadach jest ponad 30-krotnie szybsza od wariantu z pamięcią prywatną.

Głównym powodem sukcesu Wariantu 2 jest fakt, że przy rozkładzie jednostajnym i dużej liczbie kubełków ($k = n$), prawdopodobieństwo wystąpienia konfliktów (*lock contention*) jest na tyle niskie, że narzut synchronizacji jest pomijalny w porównaniu do kosztów zarządzania ogromną przestrzenią adresową i fazy scalania wymaganej w Wariancie 3. Wyniki te dowodzą, że eliminacja współdzielenia danych (Wariant 3) nie zawsze jest optymalną strategią zrównoleglania, zwłaszcza gdy narzut na zarządzanie strukturami prywatnymi rośnie wraz ze skalą problemu i liczbą procesorów.

Bibliografia

[1] IEEE and The Open Group. *drand48(3p) — Linux manual page*. Linux man-pages project. June 14, 2024. URL: `https://man7.org/linux/man-pages/man3/drand48.3p.html` (visited on 05/08/2026).